# MERN Stack Interview Q&A;

***Q1. What is the MERN stack?***

➤ The MERN stack is a collection of JavaScript-based technologies: MongoDB, Express.js, React, and Node.js.
MongoDB is the database, Express.js is the backend framework, React is the frontend library, and Node.js is the runtime environment.

***Q2. Difference between SQL and NoSQL databases?***

➤ SQL databases are relational, structured, and use schemas (e.g., MySQL, PostgreSQL).
NoSQL databases (e.g., MongoDB) are non-relational, flexible, and schema-less, suitable for large-scale applications.

***Q3. Explain MVC architecture in Express.js.***

➤ MVC stands for Model-View-Controller.
- Model: Handles data and business logic (MongoDB schema/models).
- View: Frontend/UI (React in MERN).
- Controller: Manages requests, responses, and connects Model & View.

***Q4. What is the role of Redux in React?***

➤ Redux is a state management library that helps manage global application state.
It consists of Store, Reducers, Actions, and Dispatch. It prevents prop-drilling and improves predictability.

***Q5. Difference between functional and class components in React?***

➤ Class components have lifecycle methods and state using `this.state`.
Functional components are simpler and use React Hooks (`useState`, `useEffect`, etc.), which replaced most class-based usage.

***Q6. Explain JWT Authentication flow in MERN apps.***

➤ 1. User logs in with credentials.
2. Server verifies credentials and generates JWT token.
3. Token is sent to client and stored (localStorage/cookies).
4. For protected routes, token is sent in headers.
5. Server verifies token and allows/denies access.

***Q7. What are REST APIs and their principles?***

➤ REST (Representational State Transfer) APIs follow principles like:
- Stateless communication
- Client-server separation
- Resource-based (using endpoints like /users, /products)
- HTTP methods: GET, POST, PUT, DELETE

***Q8. Difference between React useMemo and useCallback?***

➤ - `useMemo`: Memoizes the result of a calculation.
- `useCallback`: Memoizes a function reference, preventing re-creation on re-renders.

***Q9. How does React's Virtual DOM work?***

➤ Virtual DOM is a lightweight copy of the real DOM. React compares (diffs) the new Virtual DOM with the old one using the reconciliation algorithm and updates only the changed parts of the real DOM.

***Q10. Difference between cookies, localStorage, and sessionStorage?***

➤ - Cookies: Small data stored on client, sent with requests, can expire, used for auth.
- localStorage: Stores data with no expiry, persists after refresh.
- sessionStorage: Stores data for session only, clears on browser close.

***Q11. How do you optimize performance in React?***

➤ - Using React.memo
- Code splitting (React.lazy, Suspense)
- Avoiding unnecessary re-renders
- Using useMemo/useCallback
- Pagination or infinite scroll for large lists

***Q12. How to connect MongoDB in Node.js using Mongoose?***

➤ Example:
```js
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/myDB', { useNewUrlParser: true,
useUnifiedTopology: true })
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
```

***Q13. Difference between useEffect and useLayoutEffect?***

➤ - `useEffect`: Runs after render, async by default.
- `useLayoutEffect`: Runs synchronously after all DOM mutations, before painting.

***Q14. What is CORS and how to handle it?***

➤ CORS (Cross-Origin Resource Sharing) is a security feature that restricts requests from different domains.
Handled using middleware like:
```js
const cors = require('cors');
app.use(cors());
```

***Q15. Difference between Monolithic and Microservices architecture?***

➤ - Monolithic: Single large codebase, tightly coupled, harder to scale.
- Microservices: Application split into independent services, easier to scale and maintain.